

Engineering Calculator

A Unified Computational Toolkit for Vectors,
Calculus, and Numerical Methods

CSAI 101 Final Project
Zewail City of Science and Technology

Unifying the Academic Mathematical Workflow

The Challenge

Fragmented Contexts.

Engineering workflows require constant, inefficient context-switching between linear algebra, calculus analysis, and numerical solvers.

The Solution

Unified Hub.

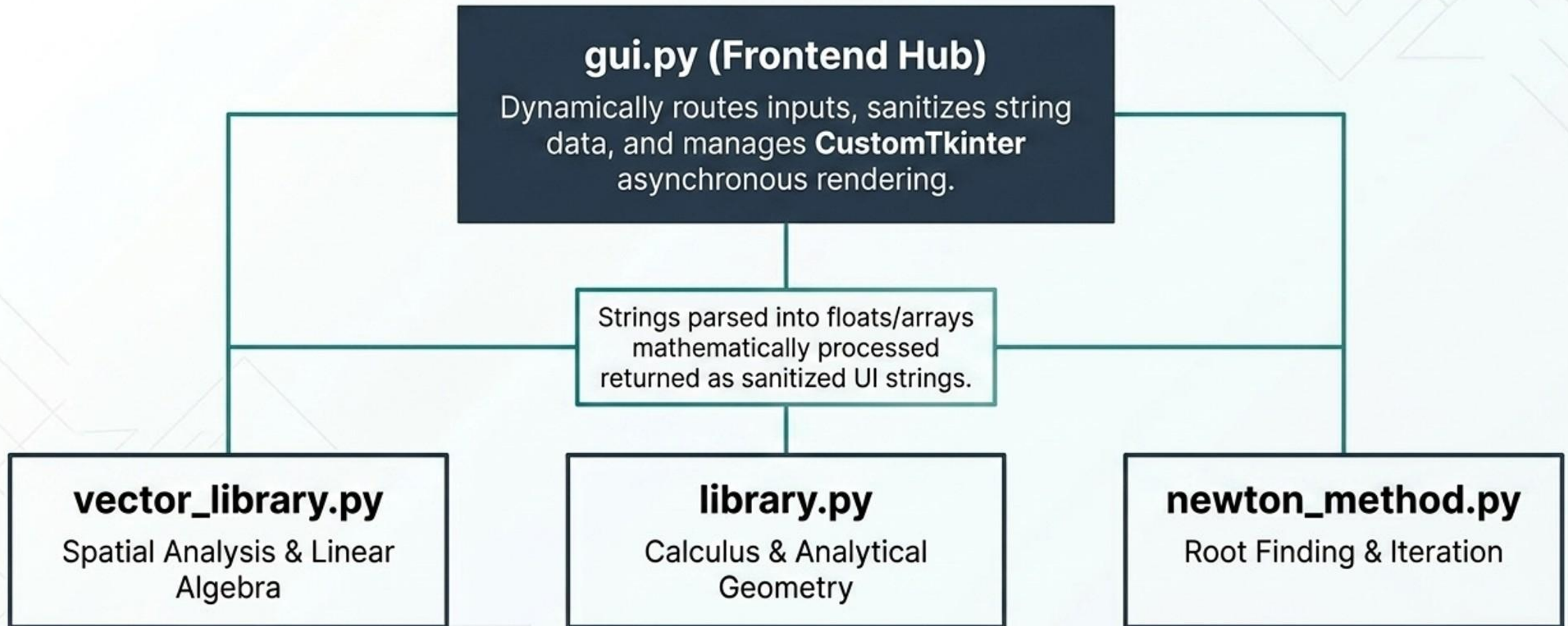
A Python-based graphical toolkit consolidating over 30 mathematical functions into a single academic dashboard.

The Standard

Strict Modularity.

Built with rigorous separation of concerns, built-in mathematical safeguards (e.g., zero-division), and dynamic error handling.

System Architecture & Separation of Concerns



Vector Operations Module (Hassan)

Dimensional Flexibility

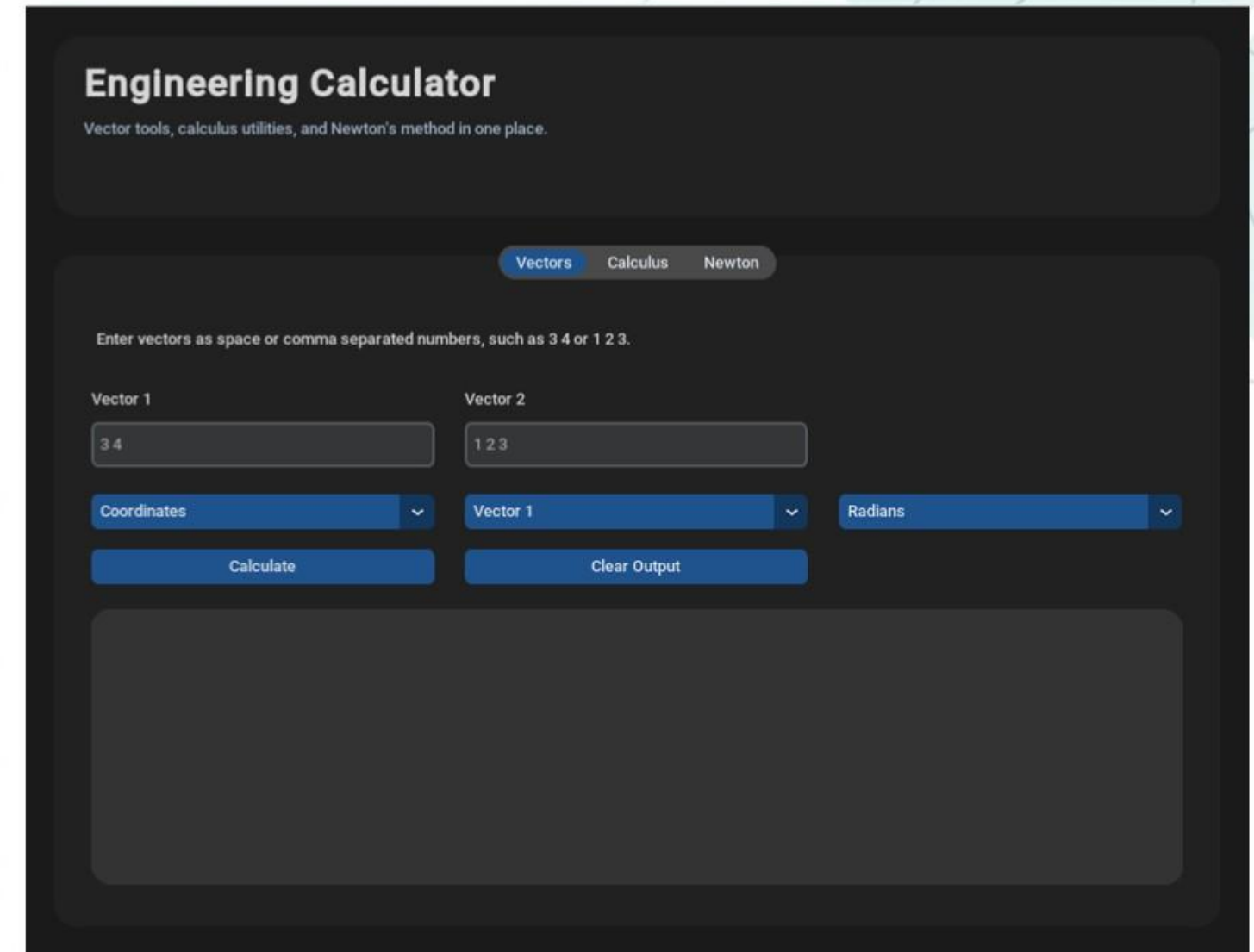
Automatically parses 2D/3D coordinates. Utilizes the `_pad_vectors` function to append 0.0 placeholder dimensions, safely evaluating planar cross products.

Spatial Analysis

Evaluates dot/cross products, structural magnitudes, and extracts inter-vector angular distances via inverse cosine operations.

Data Transformation

The `_to_numbers` function strips raw string inputs into native floats for clean linear algebra manipulation.



The screenshot shows a web application titled "Engineering Calculator" with a subtitle "Vector tools, calculus utilities, and Newton's method in one place." Below the title are three tabs: "Vectors" (selected), "Calculus", and "Newton". A instruction text says "Enter vectors as space or comma separated numbers, such as 3 4 or 1 2 3." There are two input fields: "Vector 1" containing "3 4" and "Vector 2" containing "1 2 3". Below each input field is a dropdown menu; the first is set to "Coordinates" and the second to "Vector 1". To the right of these is a third dropdown menu set to "Radians". Below the input fields are two buttons: "Calculate" and "Clear Output". A large, empty rectangular box is positioned below the buttons, intended for the output of the calculations.

Fig 1: Vector workspace executing `parse_vector` to handle comma or space-separated arrays with dimensional padding.

Calculus & Analytical Geometry Module (Mahmoud)

Definite Integrals

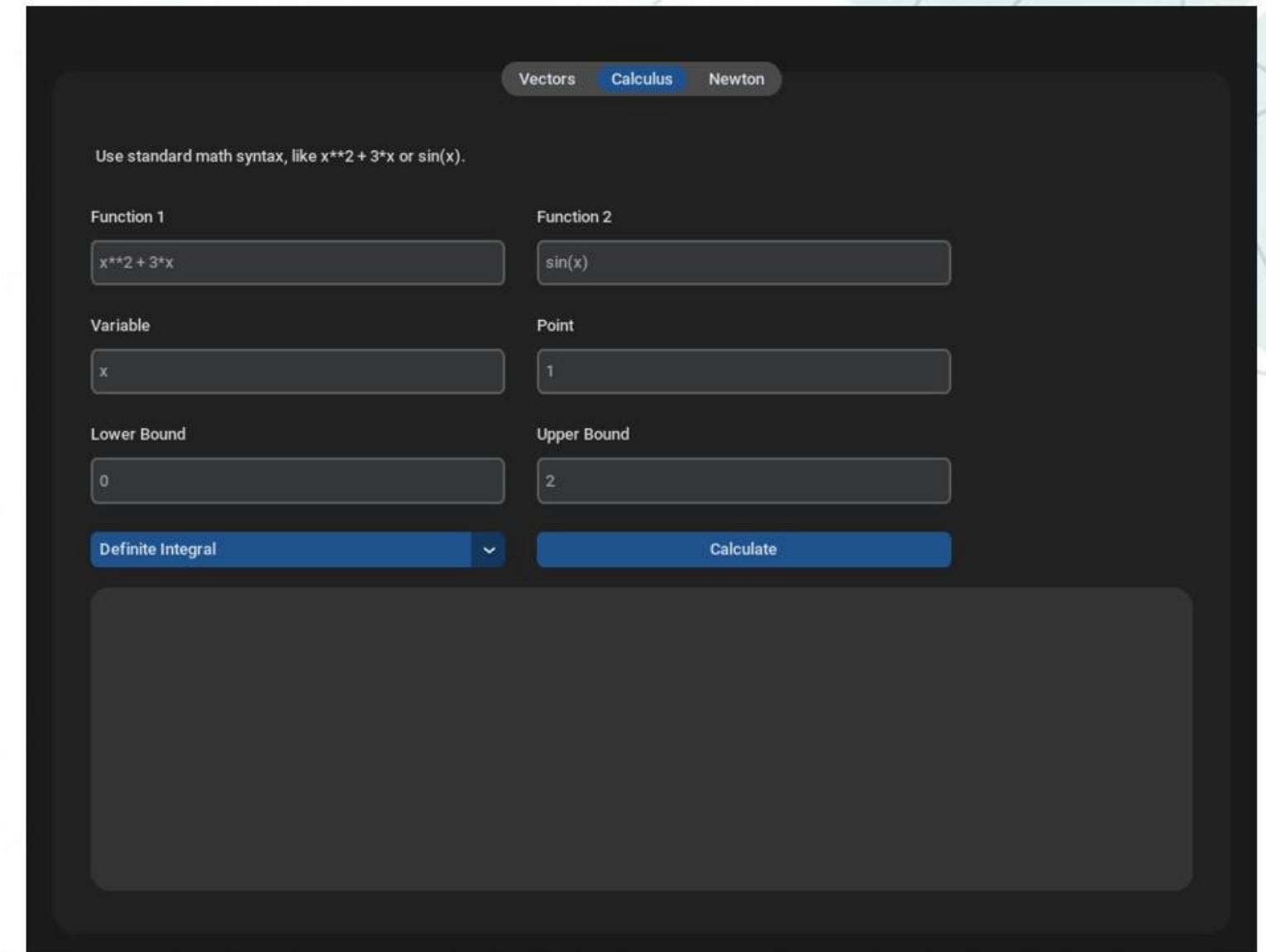
Computes single-variable bounding values utilizing standard anti-derivative parsing boundaries.

Geometric Area Analysis

area_between_2functions computes trapped area, automatically splitting integration boundaries at intersection turning points using absolute integration.

Curve Mapping

Extracts precise tangent line equations ($y = mx + c$) and normal perpendicular slopes mapping out from functional coordinate intersections.



The image shows a web-based interface for a Calculus module. At the top, there are three tabs: "Vectors", "Calculus" (which is selected and highlighted in blue), and "Newton". Below the tabs, a text prompt says "Use standard math syntax, like $x^{**2} + 3*x$ or $\sin(x)$ ". The interface is divided into two columns. The left column has input fields for "Function 1" (containing $x^{**2} + 3*x$), "Variable" (containing x), and "Lower Bound" (containing 0). The right column has input fields for "Function 2" (containing $\sin(x)$), "Point" (containing 1), and "Upper Bound" (containing 2). Below these fields, there is a dropdown menu currently set to "Definite Integral" and a blue "Calculate" button. At the bottom of the interface is a large, empty rectangular box, likely for displaying the results of the calculation.

Fig 2: Calculus environment configured for bounding integrations and algebraic curve plotting.

Numerical Methods: Newton-Raphson Iteration (Youssef)

Core Algorithm

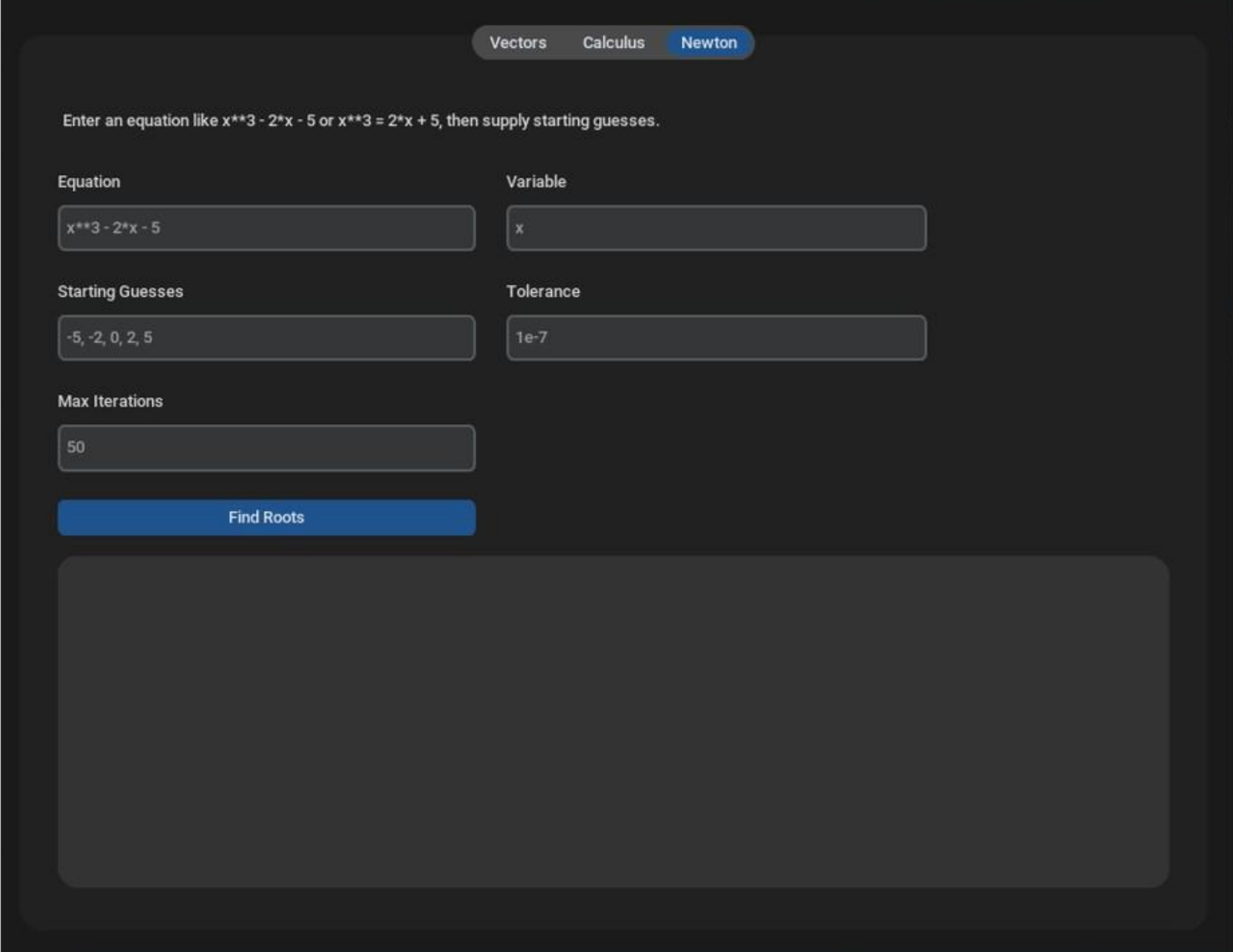
Iteratively approximates single-variable roots utilizing
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Batch Processing

The find_roots function dynamically processes multiple starting boundary guesses to isolate and compile unique convergent roots.

Mathematical Safeguards

Built-in catchers prevent zero-division errors when the derivative approaches zero, with _as_real_number rejecting unexpected complex numbers.



The image shows a web-based interface for the Newton-Raphson method. At the top, there are three tabs: "Vectors", "Calculus", and "Newton", with "Newton" being the active tab. Below the tabs, a text prompt says "Enter an equation like x**3 - 2*x - 5 or x**3 = 2*x + 5, then supply starting guesses." The interface has several input fields: "Equation" with the value "x**3 - 2*x - 5", "Variable" with the value "x", "Starting Guesses" with the value "-5, -2, 0, 2, 5", "Tolerance" with the value "1e-7", and "Max Iterations" with the value "50". A blue button labeled "Find Roots" is positioned below the input fields. At the bottom of the interface, there is a large, empty rectangular box for displaying the results.

Fig 3: Newton method interface executing multi-guess batch processing within custom user-defined tolerances.

Collaborative GUI & Responsive Dashboard Design

Unified Framework

Built utilizing CustomTkinter (`__init__`) for a responsive, dark/light mode compatible interface across 3 distinct tab views.

Input Sanitization

Standardized formatting operations (`_entry_row`, `parse_vector`) catch structural errors before backend execution triggers.

Asynchronous Rendering

`_set_textbox` cleanly wipes outdated UI metrics to display fresh execution data without interface freezing.

Output Optimization

`format_number` rounds floating-point results to a high-precision layout, automatically converting to integer variants if trailing decimals fall inside error thresholds.

Course Criteria & Compliance Verification

Compliance Checklist

Evaluation Dimension	CSAI 101 Requirement	Project Delivery
Function Volume	>15 modular functions.	30+ operational functions, ranging from UI parsing to multi-guess mathematical iteration. ✓
Modularity	Clear separation of logic.	Deep structural separation spanning 4 core Python files (gui.py, library.py, vector_library.py, newton_method.py). ✓
System Robustness	Handling edge cases.	Integrated system safeguards against unexpected complex numbers, zero-division, and malformed string inputs. ✓